

B U P T

密码工程

第9讲、国密算法及其实现 SM2算法

授课教师：王晨宇

时间：2026年4月30日

4.5

SM2算法

► SM2算法

B U P T



- ✓ 4.5.1 SM2简介
- ✓ 4.5.2 椭圆曲线
- ✓ 4.5.3 数据类型及其转换
- ✓ 4.5.4 基于SM2的公钥加密算法
- ✓ 4.5.5 基于SM2的数字签名算法
- ✓ 4.5.6 基于SM2的密钥交换协议



SM2算法

4.5.1 SM2简介

B U P T

- SM2算法由国家密码管理局于2010年12月17日发布，是我国自主设计的公钥密码算法。SM2算法在ECC椭圆曲线密码理论基础上进行自主研发设计，具备ECC算法的性能特点并实现了优化改进。
- 在商用密码体系中，SM2主要用于替换RSA加密算法。
- SM2算法是**中国国家密码局**推出的国产化算法，和RSA算法一样，同属于非对称算法体系，而且是椭圆曲线加密(ECC)算法的一种。但与RSA算法不同的是：RSA算法是基于大整数分解数学难题，SM2算法是基于椭圆曲线上点群离散对数难题。
- 基于ECC的SM2证书普遍采用256位密钥长度，加密强度等同于3072位RSA证书，远高于业界普遍采用的2048位RSA证书。



- ECC算法在工程应用中比较难以实现，但它的单位安全强度相对较高。
- 用国际上公认的针对ECC算法最有效的攻击方法——Pollard rho方法去破译和攻击ECC算法，它的破译或求解难度基本上是指数级的。

RSA密钥长度	ECC密钥长度	保密年限
2048	224	2030
3072	256	2040
7680	384	2080
15360	512	2120



SM2算法

4.5.1 SM2简介(3)

B U P T

- 相较于RSA算法，SM2算法的优点如下：
- **安全性高**:192位的SM2密码强度已经比RSA的2048位密码强度要高，存储空间小。
SM2算法的密码一般使用192~256位，RSA算法的密码一般需要2048~4096位。
- **签名速度快**:SM2算法在私钥运算上的速度远大于RSA算法。
- **国产算法**:由国家密码管理局制订规范，不存在不可公开的密码。目前普遍认为在国内被广泛使用的**RSA的1024位算法不再安全**，国家密码管理局下发了通知：自2011年7月1日起，投入运行并使用公钥密码的信息系统应使用SM2椭圆曲线公钥密码算法。



□在确保相同安全强度的前提下，SM2具有比RSA更好的性能。

算法	签名次数（次/秒）	验签次数（次/秒）
1024位RSA	2792	51224
2048位RSA	455	15122
256位SM2	4095	871



□有限域：

□包含有限数量元素的域，通常用 F_q 来表示，其中 q 表示域中的元素数量。

□当 q 是奇素数 p 时：

□此时称为素域 F_p ，在SM2算法中要求 $p > 2^{191}$ 。

□当 q 是 2 的 m 次方幂时：

□称为二元扩域 F_{2^m} ，在SM2算法中要求 $m > 192$ 且为素数。



SM2算法 4.5.2 椭圆曲线(2)

B U P T

- 对于素域 F_p ，素域中的元素用整数 $0, 1, 2, \dots, p-1$ 表示，其满足以下特性：
 - 1) 加法单位元是整数 0。
 - 2) 乘法单位元是整数 1。
 - 3) 域元素的加法是整数的模 p 加法。
 - 4) 域元素的乘法是整数的模 p 乘法。



椭圆曲线素域方程

□ 定义在 F_p 上的椭圆曲线 $E(F_p)$ 的方程为：

$$y^2 = x^3 + ax + b$$

□ 其中 $a, b \in F_p$ ，并且满足：

$$(4a^3 + 27b^2) \bmod p \neq 0$$



椭圆曲线素域上的点的运算规则

$$\square \mathbf{O} + \mathbf{O} = \mathbf{O}$$

$$\square \forall P = (x, y) \in E(F_p) \setminus \{\mathbf{O}\}, P + \mathbf{O} = \mathbf{O} + P = P$$

$$\square \forall P = (x, y) \in E(F_p) \setminus \{\mathbf{O}\}, -P = (x, -y), P + (-P) = \mathbf{O}$$



椭圆曲线素域上的点的运算规则

□点加法公式

□对于两个点 $P_1 = (x_1, y_1)$ 和 $P_2 = (x_2, y_2)$, 计算 $P_3 = P_1 + P_2$ 的公式为

$$\begin{aligned}x_3 &= \gamma^2 - x_1 - x_2 \\y_3 &= \gamma(x_1 - x_3) - y_1\end{aligned}$$

□其中 γ 为斜率, 计算公式为:

$$\gamma = \frac{y_2 - y_1}{x_2 - x_1}$$



椭圆曲线素域上的点的运算规则

□倍点运算

□对于点 $P_1 = (x_1, y_1)$ ，计算 $P_3 = P_1 + P_1$ 的公式为

$$x_3 = \gamma^2 - 2x_1$$

$$y_3 = \gamma(x_1 - x_3) - y_1$$

□其中 γ 为斜率，计算公式为：

$$\gamma = \frac{3x_1^2 + a}{2y_1}$$



二元扩域的概念

- 在二元扩域 F_{2^m} 中, 元素可以看作是 F_2 上的 m 维向量空间。
- 其元素可用长度为 m 的比特串表示。



二元扩域的性质

- 1) 零元素：零元0用全0比特串表示
- 2) 单元元素：用比特串00...001表示
- 3) 加法规则：两个域元素的加法通过比特位置异或运算进行。
- 4) 乘法规则：设 a 和 b 对应 F_2 上的多项式为 $a(x)$ 和 $b(x)$,
 - 则 $a \cdot b$ 定义为：

$$\square a(x) \cdot b(x) \pmod{f(x)}$$



椭圆曲线二元扩域方程

□ 定义在 F_{2^m} 上的椭圆曲线 $E(F_{2^m})$ 的方程为：

$$y^2 + xy = x^3 + ax + b$$

□ 其中

$$a, b \in F_{2^m}$$

□ 且

$$b \neq 0$$



生成一个椭圆曲线

- 1) 选择种子：选择长度至少为 192 比特的随机字节串作为比特串 $SEED$ 。
- 2) 计算哈希值：

$$H = Hash_{256}(SEED) = (h_{255}, h_{254}, \dots, h_0)$$

- 3) 计算 $R = \sum_{i=0}^{255} h_i 2^i$, $r = R \bmod p$
- 4) 选择 F_p 中的元素 a 和 b , 满足 $r \cdot b^2 \equiv a^3 \pmod{p}$
- 5) 若 $(4a^3 + 27b^2) \bmod p = 0$ 则重新从第1步开始计算
- 6) 输出椭圆曲线方程：

$$E: y^2 = x^3 + ax + b$$



椭圆曲线验证

□输入：为随机生成的 $SEED$ 以及参数 a, b

□1) 计算哈希值：

$$H' = Hash_{256}(SEED) = (h'_{255}, h'_{254}, \dots, h'_0)$$

□2) 计算 R'

$$R' = \sum_{i=0}^{255} h'_i 2^i, \quad r' = R' \bmod p$$

□3) 验证条件

若 $r' \cdot b^2 \equiv a^3 \pmod{p}$ ，则有效，否则无效



椭圆曲线推荐参数

- 一般情况：在加密会话中，通常双方需要预先确定一组椭圆曲线参数以及系统参数。这一过程需要大量计算，耗时较长，但一旦生成便可长期使用。
- 通常由可信机构生成不同的椭圆曲线参数并挑选出安全性和性能俱佳的参数公布为推荐参数。

SM2椭圆曲线公钥密码算法推荐曲线参数

推荐使用素数域256位椭圆曲线。
椭圆曲线方程： $y^2 = x^3 + ax + b$ 。
曲线参数：
 p =FFFFFFFF FFFFFFFF FFFFFFFF FFFFFFFF FFFFFFFF 00000000 FFFFFFFF FFFFFFFF
 a =FFFFFFFF FFFFFFFF FFFFFFFF FFFFFFFF FFFFFFFF 00000000 FFFFFFFF FFFFFFFC
 b =28E9FA9E 9D9F5E34 4D5A9E4B CF6509A7 F39789F5 15AB8F92 DDBCBD41 4D940E93
 n =FFFFFFFF FFFFFFFF FFFFFFFF FFFFFFFF 7203DF6B 21C6052B 53BBF409 39D54123
 G_x =32C4AE2C 1F198119 5F990446 6A39C994 8FE30BBF F2660BE1 715A4589 334C74C7
 G_y =BC3736A2 F4F6779C 59BDCEE3 6B692153 D0A9877C C62A4740 02DF32E5 2139F0A0



椭圆曲线的多倍点运算

□ 椭圆曲线上同一个点的多次加法运算称为该点的多倍点运算。

□ 设 k 是一个正整数, P 是椭圆曲线上的点, 则有:

$$Q = [k]P = P + P + \dots + P$$

□ Q 为 P 的 k 倍点运算



椭圆曲线上的离散对数问题

- 设有椭圆曲线 $E(F_q)$ 、阶为 n 的点 $G \in E(F_q)$ 及 $Q \in \langle G \rangle$
- 椭圆曲线上的离散对数问题是指：
 - 确定整数 $k \in [0, n - 1]$,使得 $Q = [k]G$ 。（即 Q 为 G 的 k 倍点）
 - 这意味着椭圆曲线多倍点运算可以视为一个单向函数。
- 安全性：
 - 椭圆曲线的离散对数问题直接关系到椭圆曲线加密系统的安全性，因此需要选择安全的椭圆曲线。



椭圆曲线的基点

- 椭圆曲线的基点 G 是SM2计算的核心
- 在素域上的椭圆曲线中，基点 G 满足：

$$G = (x_G, y_G) \in E(F_p), G \neq O$$

- 且 G 的阶 n 满足：

$$n > 2^{191}, \quad n > 4p^{\frac{1}{2}}$$



椭圆曲线的抗攻击设计——MOV攻击

□定义：

- MOV攻击将有限域 F_q 上的椭圆曲线的离散对数问题转化为有限域 F_{q^B} ($B > 1$)中的离散对数问题。攻击者可以通过这种方式破解加密密钥。

□例子：

- 假设你有一把锁，传统的破解方法是尝试每一个密码，直到成功。
- 而MOV攻击通过“简化”密码的破解过程，找到一个更易破解的“弱锁”，从而通过较低的计算复杂度破解锁的密码。



椭圆曲线的抗攻击设计——MOV攻击

□ 如何防范MOV攻击：

□ 为了避免MOV攻击，椭圆曲线的阶 q 必须满足：

$$q > 2^{191} \text{ 且 } B \geq 27$$

□ 也就是说，攻击者要找到弱锁（较小的阶）非常困难，防止了破解。



椭圆曲线的抗攻击设计——异常曲线攻击

□定义：

- 如果椭圆曲线的阶 $E(F_p)$ 和素数 p 相等，攻击者可以利用这个信息快速破解加密密钥。

□例子：

- 想象一个谜题：你被告知密码的长度等于谜底的数量。
- 正常的加密方式应该避免这种简单的关系，确保谜底不是直接等于密码长度。



椭圆曲线的抗攻击设计——异常曲线攻击

□ 如何防范抗异常曲线攻击：

- 为了避免这种攻击，我们要求椭圆曲线的阶 $E(F_p)$ 不等于素数 p ，即确保密码长度和谜底之间没有直接的线性关系。



椭圆曲线的抗攻击设计——Pohlig-Hellman 和 Pollard攻击

□定义:

- 这些攻击方法依赖于椭圆曲线的基点 G 选择太小，导致攻击者可以通过简单的算法找到基点的离散对数，从而破坏加密系统。

□例子:

- 假设你有一把非常简单的密码锁，只有几个选择。
- 攻击者就能很轻松地试出所有密码，而这正是这些攻击方法的基本原理。



椭圆曲线的抗攻击设计——Pohlig-Hellman 和 Pollard攻击

□ 如何防范Pohlig-Hellman 和 Pollard攻击：

- 为了避免这类攻击，椭圆曲线的基点 G 应该是一个非常大的素数，使得攻击者无法通过简单的算法破解密码。



数据类型

□ 在SM2算法中，数据包括以下几种类型：

□ 比特串：由0/1序列构成。

□ 字节串：由字节序列组成，每8比特组成一个字节。

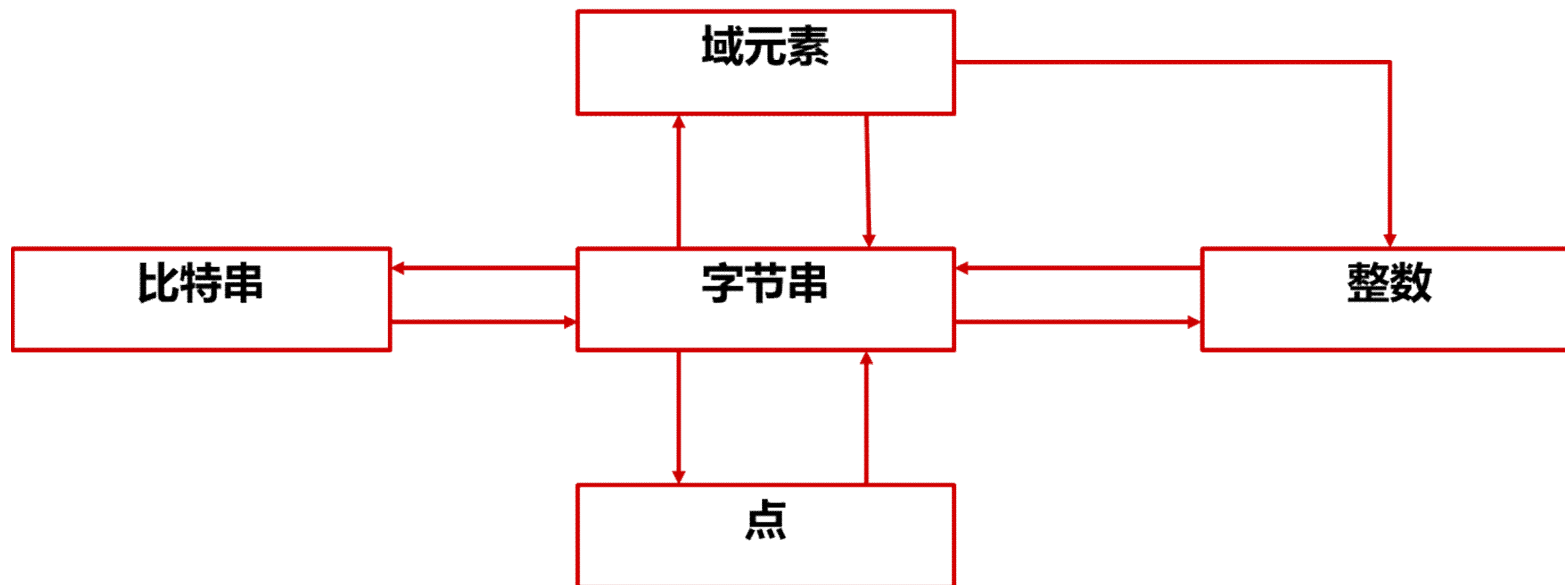
□ 域元素：有限域 F_q 中的元素。

□ 椭圆曲线上的点：记为 $T = (x_t, y_t) \in E(F_q)$ ，其中 T 是椭圆曲线上的点，或者是无穷远点 O 。



数据类型转换

□ 不同类型的数据可以互相转换。例如，比特串可以转换为字节串，字节串可以转换为整数等。具体转换规则如下：





整数与字节串的转换

□ 设字节串 M 从左到右的字节为 $M_{k-1}, M_{k-2}, \dots, M_0$ ，则 M 和整数 x 之间的转换公式为：

$$x = \sum_{i=0}^{k-1} 2^{8i} M_i$$

□ 其中，每个字节 M_i 被转换为其对应的8位二进制表示。



比特串与字节串的连接

- 设比特串 s 长度为 m , 字节串 M 长度为 k , 则 $k = \left\lceil \frac{m}{8} \right\rceil$, 每8个比特组成一个字节。
- 比特串: $s_{m-1}, s_{m-2}, \dots, s_0$
- 字节串: $M_{k-1}, M_{k-2}, \dots, M_0$
- 转换公式: $M_i = s_{8i+7} s_{8i+6} \dots s_{8i+1} s_{8i}$
- 即从右往左, 每8个比特组成一个字节, 若高位不满8位则补零。



域元素与字节串的连接

- 假设域 F_q 中的域元素 a ，则字节串的长度为 $l = \left\lceil \frac{\lceil \log_2 q \rceil}{8} \right\rceil$ 。
- 如果 q 为奇素数，则将 a 转换为整数 x ，再将其转换为字节串。
- 如果 $q = 2^m$ ，则直接将其转换为比特串，再将比特串转换为字节串。



域元素与字节串的转换

□ 例如， q 为奇素数时：

□ $p = 65537$ （16 位素数），域元素 $a = 10000$

□ 计算字节长度 l ：

$$l = \left\lceil \frac{\lceil \log_2 q \rceil}{8} \right\rceil = \left\lceil \frac{\lceil 16 \rceil}{8} \right\rceil = 2 \text{ 字节}$$

□ 把 a 先视为整数再转 2 字节串

$$a = 10000_{10} = 0x2710_{16} \Rightarrow [0x27, 0x10]$$



域元素与字节串的转换

□ 例如, $q = 2^m$ 时:

□ 域元素 $a = 10000 = 1011\ 0110_2$

□ 直接写成字节:

$$1011\ 0110_2 = 0xB6_{16}$$



椭圆曲线上的点转换为字节串

- 给定椭圆曲线上的点 $P = (x_p, y_p)$ ，且 $P \neq O$ ，将其转换为字节串。
- 字节串表示有三种方式：
 - 压缩表示、未压缩表示、混合表示。
- 在此采用未压缩表示。
- 将域元素 x_p, y_p 转换为长度为 $l = \left\lceil \frac{\log_2 q}{8} \right\rceil$ 的字节串 X_p, Y_p 。
- 字节串 $S = 04|X_p|Y_p$ ，长度为 $2l + 1$



字节串转换为椭圆曲线上的点

- 检验字节串最左边的一个字节。
- 02/03(压缩表示)、04(未压缩表示)、06、07(混合表示)。



密钥派生函数

□目的：密钥派生函数用于从一个共享的秘密值（如密码或随机数）生成密钥。
表示为 $KDF(Z, klen)$ 。

□1、设置初始参数：

□ $i = 1$ ：迭代计数器

□ $ct = 0$ ：计数器

□ $t = \left\lceil \frac{klen}{v} \right\rceil$ ：确定总的迭代次数

□ $k l e n$ ：需要生成的密钥长度

□ v ：哈希输出的长度

While $i \leq t$:

$$H_i = Hash_v(Z|ct)$$

$ct++$

$$K = H_1|H_2|.....|H_{t-1}|H_t$$



密钥派生函数

□2、生成密钥：

□在 t 次迭代内，生成多个哈希值。

□其中

□ Z 是输入的秘密值。

□ ct 是当前计数器。

While $i \leq t$:

$$H_i = Hash_v(Z|ct)$$

$ct++$

$$K = H_1|H_2.....|H_{t-1}|H_t,$$

□3、更新计数器：

□每次迭代后，增加计数器 ct ，然后继续生成下一个哈希值。



密钥派生函数

□4、合并结果：

□将所有生成的哈希值合并成最终的密钥。

□ $H_{t'}$ 为 H_t 的前 n 个比特，可使密钥长度正好为 $klen$ 。

While $i \leq t$:

$$H_i = Hash_v(Z|ct)$$

$ct++$

$$K = H_1|H_2.....|H_{t-1}|H_{t'}$$



加密算法

□ 输入：明文 M ，长度为 $klen$ 。

□ 1、生成随机数：

□ 生成 $k \in [1, n - 1]$ 用于后续加密。

□ 2、计算椭圆曲线点1：

$$C_1 = [k]G = (x_1, y_1)$$

□ G 是椭圆曲线的基点

□ $[k]G$ 是基点 G 乘以随机数 k 的结果。



加密算法

□3、计算椭圆曲线上的点2：（ P_B 是接收方的公钥。）

$$[k]P_B = (x_2, y_2)$$

□4、加密消息 M ：

$$t = KDF(x_2|y_2, klen)$$

$$C_2 = M \oplus t$$

□5、计算哈希：

$$C_3 = Hash(x_2|M|y_2)$$

□输出： $C = C_1|C_2|C_3$



解密算法

□ 输入: $C = C_1|C_2|C_3$

□ 1、验证椭圆曲线点:

□ 验证 C_1 是否为椭圆曲线上的点。

□ 2、计算椭圆曲线上的点:

$[d_B]C_1 = (x_2, y_2)$, 其中 d_B 为私钥。

□ 3、密钥派生:

$$t = KDF(x_2|y_2, klen)$$



解密算法

□4、解密消息：

$$M = C_2 \oplus t$$

□5、验证哈希值：

$$u = \text{Hash}(x_2 | M | y_2)$$

□验证 $u = C_3$ ，如果不等，则解密失败。

□输出：明文消息 M 。



数字签名算法

- 数字签名算法由一个签名者对消息生成数字签名，并由一个验证者验证签名的可靠性。
- 每个签名者有一个公钥和一个私钥。
 - 私钥用于产生签名。
 - 公钥用于验证签名。



数字签名算法

□ 签名者 A 通常包含以下信息：

□ 签名者的身份 ID_A ：长度为 $entlen_A$ 比特。

□ 由 $entlen_A$ 转换成的字节串 $ENTL_A$ 。

□ 椭圆曲线参数：包括曲线系数 a, b 以及基点 (x_G, y_G) 。

□ A 的公钥： (x_A, y_A) ，私钥： d_A 。

□ 签名者 A 的Hash值： $Z_A = Hash_{256}(ENTL_A|ID_A|a|b|x_G|y_G|x_A|y_A)$



数字签名生成算法

□ 设待签名的消息为 M ，生成的数字签名为 (r, s) 。

□ 1、计算随机数：

□ 计算哈希值 $e = H_v(Z_A || M)$ 。

□ 计算随机点 $(x_1, y_1) = [k]G$ ，其中 k 为随机数。

□ 计算：

$$r = (e + x_1) \bmod n$$

□ 如果 $r = 0$ 或 $r + k = n$ ，则重新选择随机数。



数字签名生成算法

□ 设待签名的消息为 M ，生成的数字签名为 (r, s) 。

□ 2、计算签名值 s ：

$$s = ((1 + d_A)^{-1} \cdot (k - r \cdot d_A)) \bmod n$$

□ 若 $s = 0$ ，重新选择随机数。

□ 3、转换为字节串：

□ 将 (r, s) 转换为字节串。



数字签名验证算法

□1、接收到的签名

□验证接收到的数字签名 (r, s) 是否在 $[1, n - 1]$ 的范围内。

□2、计算哈希值

□计算 $e = H_v(Z_A || M)$ 。



数字签名验证算法

□3、验证

$$t = (r + s) \bmod n \neq 0$$

□计算：

$$(x, y) = [s]G + [t]P_A$$

□验证：

$$(e + x) \bmod n = r$$

□如果成立，签名有效；否则，签名无效。



1.在SM2算法中，生成和验证数字签名是非常关键的一步。如果某一方生成的数字签名不合法，另一方如何检测并防止错误的签名被使用？

请结合你对数字签名算法的理解，简要说明数字签名的验证过程及其对安全通信的重要性。



协议概述

- 在该协议中，发起方 A 和 响应方 B 通过一系列的计算步骤实现密钥的安全交换。
- 通过这些步骤，双方能够在没有事先共享密钥的情况下，计算出相同的对称密钥（通常称为共享密钥），这个密钥将在后续的加密通信中使用。



SM2算法 4.5.5 基于SM2的密钥交换协议(2)

B U P T

发起方用户A

Step1: 产生随机数 $r_A \in [1, n - 1]$

Step2: $R_A = [r_A]G = (x_1, y_1)$

Step3: $\bar{x}_1 = 2^w + (x_1 \& (2^w - 1))$

Step4: $t_A = (d_A + \bar{x}_1 \cdot r_A) \bmod n$

响应方用户B

Step1: 产生随机数 $r_B \in [1, n - 1]$

Step2: $R_B = [r_B]G = (x_2, y_2)$

Step3: $\bar{x}_2 = 2^w + (x_2 \& (2^w - 1))$

Step4: $t_B = (d_B + \bar{x}_2 \cdot r_B) \bmod n$

$\xrightarrow{R_A}$ 确认 R_A 满足曲线方程

- Step2: 计算椭圆曲线上的点 R_A / R_B
- Step3: 对椭圆曲线点 R_A / R_B 的 x_1 / x_2 坐标进行操作
- Step4: 根据私钥 d_A / d_B 计算 t_A / t_B



SM2算法 4.5.5 基于SM2的密钥交换协议(3)

B U P T

发起方用户A

Step5: $\bar{x}_2 = 2^w + (x_2 \& (2^w - 1))$

Step6: $U = [h \cdot t_A](P_B + [\bar{x}_2]R_B) = (x_U, y_U)$

确认 $U \neq O$

Step7: $K_A = KDF(x_U || y_U || Z_A || Z_B, klen)$

Step8: $S_1 = Hash(0x02 || y_U ||$
 $Hash(x_U || Z_A || Z_B || x_a || y_a || x_b || y_b))$

确认 $S_1 = S_B$

响应方用户B

Step5: $\bar{x}_1 = 2^w + (x_1 \& (2^w - 1))$

Step6: $V = [h \cdot t_B](P_A + [\bar{x}_1]R_A) = (x_V, y_V)$

确认 $V \neq O$

Step7: $K_B = KDF(x_V || y_V || Z_A || Z_B, klen)$

Step8: $S_B = Hash(0x02 || y_V ||$
 $Hash(x_V || Z_A || Z_B || x_a || y_a || x_b || y_b))$

R_B, S_B

- Step5: A和B分别对所接收到的 R_A/R_B 进行类似Step3的计算。
- Step6: 生成共享密钥的一部分 U/V
- Step7: 使用密钥派生函数各自生成一个共享密钥 K_A/ K_B
- Step8: 计算共享的哈希值 S_1 / S_B , A 验证生成的共享密钥是否与 B 一致。



SM2算法 4.5.5 基于SM2的密钥交换协议(4)

B U P T

发起方用户A

响应方用户B

Step9: $S_A = \text{Hash}(0x03 || y_U ||$
 $\text{Hash}(x_U || Z_A || Z_B || x_a || y_a || x_b || y_b))$

Step9: $S_2 = \text{Hash}(0x03 || y_V ||$
 $\text{Hash}(x_V || Z_A || Z_B || x_a || y_a || x_b || y_b))$

$\xrightarrow{S_A}$ 确认 $S_2 = S_A$

□ Step9: 计算共享的哈希值 S_A / S_2 , B 验证生成的共享密钥是否与 A 一致。



生成公私钥对

□使用以下命令生成私钥：

□`gmssl sm2 -genkey -sms4 -out sm2_key.pem`

□`sm2`：指定使用 SM2 算法。

□`-genkey`：表示生成密钥。

□`-sms4`：使用 SMS4 算法。

□`-out sm2_key.pem`：指定输出密钥文件的名称为 `sm2_key.pem`，文件中将存储 SM2 的私钥。



生成公私钥对

□生成私钥:

□`gmssl sm2 -genkey -sms4 -out sm2_key.pem`

```
lsy@ubuntu:~$ gmssl sm2 -genkey -sms4 -out sm2_key.pem
Using configuration from /usr/local/ssl/openssl.cnf
generate SM2 key
writing SM2 key
Enter PEM pass phrase:
Verifying - Enter PEM pass phrase:
```



生成公私钥对

□ 使用以下命令从私钥中导出公钥：

□ `gmssl sm2 -in sm2_key.pem -pubout -out sm2_pub.pem`

□ **-in sm2_key.pem**：指定输入文件是之前生成的私钥文件 `sm2_key.pem`。

□ **-pubout**：表示输出公钥。

□ **-out sm2_pub.pem**：指定输出的公钥文件名为 `sm2_pub.pem`。



生成公私钥对

□ 使用以下命令从私钥中导出公钥：

□ `gmssl sm2 -in sm2_key.pem -pubout -out sm2_pub.pem`

```
lsy@ubuntu:~$ gmssl sm2 -in sm2_key.pem -pubout -out sm2_pub.pem
Using configuration from /usr/local/ssl/openssl.cnf
read SM2 key
Enter PEM pass phrase:
writing SM2 key
```



公钥加密

□ 使用公钥加密文件：

□ gmssl **sm2utl** -encrypt -in test.txt -pubin -inkey sm2_pub.pem -out encryption.der

□ 注：

□ sm2 主要用于密钥相关的操作，如密钥对生成和导出公钥。

□ sm2utl 则是一个更全面的工具，支持加密、解密、签名和验证签名等操作。

□ 如前文的导出公钥： gmssl **sm2** -in sm2_key.pem -pubout -out sm2_pub.pem



公钥加密

□ 使用公钥加密文件：

□ `gmssl sm2utl -encrypt -in test.txt -pubin -inkey sm2_pub.pem -out encryption.der`

□ **-encrypt**：指定执行加密操作。

□ **-in test.txt**：指定输入文件为 `test.txt`，即需要加密的文件。

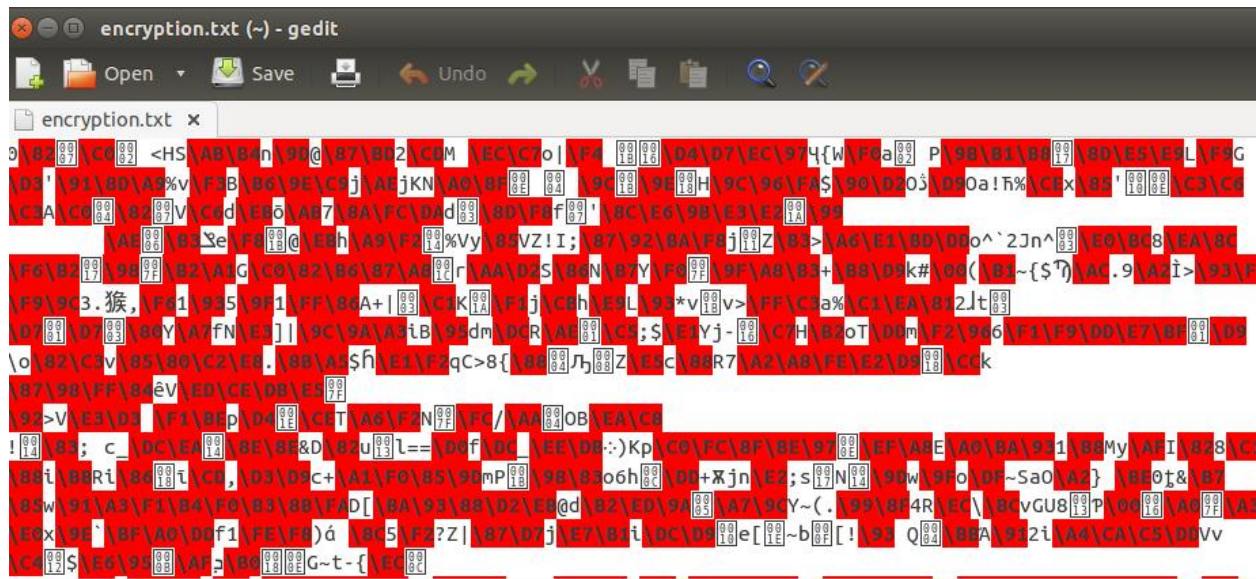
□ **-pubin**：表示输入的是公钥。

□ **-inkey sm2_pub.pem**：指定公钥文件为 `sm2_pub.pem`。

□ **-out encryption.der**：指定输出的加密结果保存为 `encryption.der`。



- 使用公钥加密文件:
- `gmssl sm2utl -encrypt -in test.txt -pubin -inkey sm2_pub.pem -out encryption.der`





私钥解密

□ 使用私钥解密文件：

□ `gmssl sm2utl -decrypt -in encryption.txt -inkey sm2_key.pem -out decryption.txt`

□ **-decrypt**：指定执行解密操作。

□ **-in encryption.txt**：指定输入的加密文件为 `encryption.txt`（可以是 `encryption.der`）。

□ **-inkey sm2_key.pem**：指定私钥文件为 `sm2_key.pem`。

□ **-out decryption.txt**：指定解密结果输出到 `decryption.txt` 文件。



私钥解密

□ 使用私钥解密文件：

□ `gmssl sm2utl -decrypt -in encryption.txt -inkey sm2_key.pem -out decryption.txt`

```
decryption.txt (~) - gedit
Open Save Undo
decryption.txt x
这是一个测试文本，里面包含了很多信息：
GmSSL是一个开源的密码工具包，它为GM/T系列标准中规定的中国国家密码算法和协议提供一级支持。作为OpenSSL项目的一个分支，GmSSL提供了与OpenSSL的API级兼容性，并维护了所有的功能。现有的项目（如apacheweb服务器）可以轻松地移植到GmSSL，只需稍加修改和简单的重建。自2014年底首次发布以来，GmSSL被选为开源中国推荐的六大加密项目之一，并荣获2015年中国Linux软件奖。
GmSSL will support all the following GM/T cryptographic algorithms:

SM3 (GM/T 0004-2012): cryptographic hash function with 256-bit digest length.
SM4 (GM/T 0002-2012): block cipher with 128-bit key length and 128-bit block size, also named SMS4.
SM2 (GM/T 0003-2012): elliptic curve cryptographic schemes including digital signature scheme, public key encryption, (authenticated) key exchange protocol and one recommended 256-bit prime field curve sm2p256v1.
SM9 (GM/T 0044-2016): pairing-based cryptographic schemes including identity-based digital signature, encryption, (authenticated) key exchange protocol and one 256-bit recommended BN curve.
ZUC (GM/T 0001-2012): stream cipher, with 128-EEA3 encryption algorithm and 128-EIA3 integrity algorithm.
```




消息签名

□ 使用私钥对消息签名：

□ `gmssl sm2utl -sign -in test.txt -inkey sm2_key.pem -id Alice -out sig.txt`

□ **-sign**：表示进行签名操作。

□ **-in test.txt**：指定输入文件 `test.txt`，即需要签名的文件。

□ **-inkey sm2_key.pem -id Alice**：指定使用的私钥文件 `sm2_key.pem`，私钥用于生成数字签名，且签名者为Alice。

□ **-out sig.txt**：指定输出文件为 `sig.txt`，该文件将保存生成的数字签名。



消息签名

□ 使用公钥验证签名：

□ `gmssl sm2utl -verify -in test.txt -sigfile sig.txt -pubin -inkey sm2_pub.pem -id Alice`

□ **-verify in test.txt**：表示验证签名，输入文件 test.txt。

□ **-sigfile sig.txt**：sig.txt包含要验证的数字签名。

□ **-pubin inkey sm2_pub.pem**：表示输入的是公钥，公钥文件为 sm2_pub.pem 。

□ **-id Alice**：指定签名者的标识符，与生成签名时的标识符一致，这里是 "Alice"。



消息签名

□ 使用公钥验证签名：

□ `gmssl sm2utl -verify -in test.txt -sigfile sig.txt -pubin -inkey sm2_pub.pem -id Alice`

```
lsy@ubuntu:~$ gmssl sm2utl -verify -in test.txt -sigfile sig.txt -pubin -inkey sm2_pub.  
pem -id Alice  
Signature Verification Successful
```

[在Node js中生成国密SM2密钥对](#)



2.在SM2密钥交换协议中，双方都需要使用公钥来交换信息并确保最终生成相同的共享密钥。请结合实际应用，思考一下在多方通信中，如果某一方的公钥被篡改，可能会导致哪些安全问题？你认为如何可以防止这种攻击？

谢谢大家
